Session 19 — Study Guide: Prisma, The First Migration

Where the session ends

This session opens Block 5 (ORM/PostgreSQL migration). You now have a real `schema.prisma` describing `User` / `Workspace` / `WorkspaceMember` / `Project` / `Issue`, each migrated for real against a real PostgreSQL database, a real injectable `PrismaService`, and real routes proving queries at up to three relation levels deep — and every existing Mongoose-backed route is still exactly as it was.

The real starting point: a short, bounded recap, not a re-teach

Class opens with a brief recap of the final relational schema you already built by hand across Sessions 17-18 — not a gate on Session 18's own homework completion, which stays individual follow-up. The only real prerequisite today is the schema itself, already in your head.

What an ORM actually solves (never "a nicer SQL syntax")

Three genuinely different things: `schema.prisma` *describes* the intended database shape; a *migration* is a real, generated, versioned SQL file that actually changes the real database to match it; Prisma Client is the generated, type-safe API your application code calls to query. All three are real, inspectable artifacts, never one blended concept.

Authoring the schema — the same design, a different language

PRISMA

```
model User {
  id           Int      @id @default(autoincrement())
  name         String
  email        String   @unique
  passwordHash String   @map("password_hash")
  createdAt    DateTime @default(now()) @map("created_at")

  @@map("users")
}
```

Every line corresponds to something you already wrote by hand in `sql-practice/schema.sql`: `@id @default(autoincrement())` is `SERIAL PRIMARY KEY`; `@unique` is `UNIQUE`; `@map` / `@@map` keep the real snake_case SQL names matched to Prisma's own camelCase convention.

Relations and delete behavior — Prisma's own real default, proven

Writing nothing for `onDelete` does not mean "no behavior" — it selects Prisma's own real default, confirmed by inspecting the generated SQL: `ON DELETE RESTRICT ON UPDATE CASCADE`. This is exactly right for `Workspace.owner` (a user who still owns a workspace should never be deletable without first transferring ownership — the real transaction from Session 18's own homework). `WorkspaceMember` writes `onDelete` explicitly on both of its own relations, and deliberately not the same value for both: `workspace_id` is `Cascade` (Block 4's own ownership-hierarchy category); `user_id` is `NoAction` — reproducing, exactly, the real bare default Block 4's own delivered SQL already left on that column.

`prisma migrate dev` — a real migration, read line by line

TEXT

```
CREATE TABLE "users" ( ... );
ALTER TABLE "workspaces" ADD CONSTRAINT "workspaces_owner_id_fkey"
  FOREIGN KEY ("owner_id") REFERENCES "users"("id") ON DELETE RESTRICT ON UPDATE CASCADE;
```

Real output, generated and actually applied to a real local PostgreSQL database — tracked afterward in Prisma's own real `_prisma_migrations` table. Nothing here is magic; it is the exact `CREATE TABLE` / `ALTER TABLE` / `FOREIGN KEY` vocabulary from Block 4, generated instead of typed.

Teamwork: repeat the mechanism, then learn one real new thing

Your team added `Project` by repeating the exact mechanism independently, then researched `include` — fetching a parent together with its related records in one real query, proven two levels deep (`Workspace` → `members` → `user`), and proved the pre-existing Mongoose route was completely unaffected.

Homework: a third resource, deeper still

`Issue` with real SET NULL delete behavior; a three-level relation query; and two written design questions about what this session deliberately left out.

Common beginner mistakes

- Treating Prisma as a nicer way to write SQL — it is a description, a real migration, and a real query API, three different things.
- Forgetting a second relation to the same model needs its own name (`@relation("Name", ...)`).
- Making a field required and then trying `onDelete: SetNull` on it — Prisma will reject this.
- Confusing `include` (adds related records) with `select` (replaces which scalar fields come back).

A preview of what's still missing

Nothing real was migrated today — a brand-new, empty database, proving the mechanism safely. The real issue-key atomic-counter mechanism (Block 4's own transactions, expressed through Prisma) and the real step of copying actual MongoDB data into PostgreSQL are both deliberately next, not forgotten.

Not yet

Prisma's own transaction and raw-SQL-interop mechanisms; the real atomic issue-key counter mechanism expressed in Prisma; copying real MongoDB data into PostgreSQL; switching any existing route from Mongoose to Prisma; any frontend framework; Prisma's own newer configuration-file/CLI workflow (this project is deliberately pinned to an older, classic CLI version).

Glossary

- **Migration** — a real, generated, versioned SQL file that changes a real database to match `schema.prisma`.
- **`@map` / `@@map`** — keep a real SQL column/table name distinct from its Prisma field/model name.
- **Relation** — Prisma's declarative way of describing a foreign key.
- **`include`** — fetches related records alongside a parent in one query, nestable to any depth.
- **`onDelete`** — the real delete-behavior choice for a relation; Prisma's own default, when none is written, is `RESTRICT ON UPDATE CASCADE`.

Review questions

1. What are `schema.prisma`, a migration, and Prisma Client — one sentence each?
2. Why is Prisma's own default delete behavior on `Workspace.owner` the *correct* choice, not an accident?
3. What does `_prisma_migrations` actually track?
4. What, specifically, does `include` add to a query result that wasn't there before?

Answers

1. `schema.prisma` describes the intended database shape; a migration is the real, generated SQL that changes the real database to match it; Prisma Client is the generated API used to query it.
2. A workspace must always have an owner — `RESTRICT` prevents deleting a user who still owns one, forcing an explicit ownership transfer first, exactly the real transaction already practiced in Session 18's homework.
3. Which real migrations have actually been applied to this specific real database, by checksum — Prisma's own real audit trail.
4. The related records themselves (e.g. a workspace's own `owner` / `members` / `projects`) — without `include`, only the parent's own scalar fields come back; the related data is entirely absent, not merely less detailed.

What you should be able to explain now

What a migration actually is, as a real file you can read. Why Prisma's own default delete behavior is a real, deliberate choice, not an accident. How `include` changes what a query actually returns. Why nothing about this session put any existing Mongoose-backed route at risk.